



PROJE TASARIM RAPORU

Ders: BİL 204 / Yazılım Mühendisliği

Bölüm/Şube: Bilgisayar Mühendisliği

Grup Adı: Datariders

Proje Adı: Spor Salonu ve Tesis Yönetim Sistemi

Teslim Tarihi: 04.05.2026

Grup Üyeleri:

- İlayda Kuru (23100011015)
- Fatma Ravza Sayim (24100011077)
- Adnan Coşkun (25100011468)
- Ayşe Büşra Çamalan (24100011059)
- Nesrin Yapalı (24100011058)
- Süleyman Özkul (24100011035)

İçindekiler

1. Giriş.....	2
1.1. Sistemin Amacı.....	3
1.2. Tasarım Hedefleri.....	3
1.2.1. Kullanılabilirlik (Usability).....	3
1.2.2. Güvenilirlik ve Veri Bütünlüğü (Reliability).....	3
1.2.3. Bakım Yapılabilirlik (Maintainability).....	3
2. Üst Düzey Yazılım Mimarisi.....	3
2.1. Alt Sistem Ayırıştırması.....	3
2.2. Donanım/Yazılım Eşlemesi.....	4
2.3. Kalıcı Veri Yönetimi.....	4
2.4. Erişim Kontrolü ve Güvenlik.....	4
2.5. Küresel Kontrol Akışı.....	4
2.6. Sınır Koşulları.....	4
2.6.1. Başlatma.....	4
2.6.2. Sonlandırma.....	5
2.6.3. Hata Durumları.....	5
3. Alt Düzey Tasarım.....	5
3.1. Nesne Tasarım Kararları.....	5
3.2. Son Nesne Tasarımı.....	5
3.3. Paketler / Katmanlar.....	5
3.3.1. Dış Paketler.....	5
3.3.2. İç Paketler.....	6
3.4. Sınıf Arayüzleri.....	6
3.5. Tasarım Desenleri.....	6
Sözlük (Glossary).....	7
Referanslar.....	7

1. Giriş

1.1. Sistemin Amacı

Spor Salonu ve Tesis Yönetim Sistemi projesi, spor salonları ve tesislerin günlük operasyonlarını (rezervasyon, ders planlama, etkinlik takibi, üye yönetimi) dijitalleştirerek tek bir bilgi sistemi üzerinden yönetilmesini amaçlamaktadır. Manuel yürütülen süreçlerin getirdiği zaman kaybı ve karışıklıkları önleyerek daha düzenli, verimli ve kontrol edilebilir bir yönetim altyapısı sunmayı hedefler.

1.2. Tasarım Hedefleri

Projenin temel kalite hedefleri (non-fonksiyonel gereksinimleri) aşağıda detaylandırılmıştır:

1.2.1. Kullanılabilirlik (Usability)

Sistem, teknik bilgi düzeyi düşük kullanıcıların ve işletme personelinin rahatlıkla kullanabileceği sade, anlaşılır ve yönlendirici bir grafiksel kullanıcı arayüzüne sahip olacaktır.

1.2.2. Güvenilirlik ve Veri Bütünlüğü (Reliability)

Rezervasyon çakışmalarını önlemek ve ödeme işlemlerinde veri tutarsızlığının önüne geçmek temel hedeftir. Çakışan rezervasyon talepleri veritabanı aşamasından önce engellenecektir.

1.2.3. Bakım Yapılabilirlik (Maintainability)

Sistem, katmanlı mimari prensiplerine (Arayüz, İş Mantığı, Veri Erişimi) uygun tasarlanarak kodun modüler yapıda olması ve gelecekteki değişikliklerin kolay entegre edilmesi sağlanacaktır.

2. Üst Düzey Yazılım Mimarisi

2.1. Alt Sistem Ayırıştırması

Sistem işlevsel sorumluluklarına göre çeşitli alt paketlere/modüllere ayrıştırılmıştır:

- **Kullanıcı Yönetim Modülü:** Üye ve antrenör kayıtlarının, giriş/çıkış ve profil güncellemelerinin yapıldığı modül.
- **Rezervasyon ve Tesis Modülü:** Salon, halı saha, basketbol sahası gibi fiziksel tesislerin müsaitlik durumunu denetler ve rezervasyonları yönetir.
- **Ders ve Etkinlik Modülü:** Pilates, Yoga gibi grup derslerinin ve turnuva/kamp gibi etkinliklerin planlanması ve katılım takiplerini sağlar.
- **Ödeme ve Üyelik Modülü:** Üyelik paketi satın alma, makbuz oluşturma ve ödeme durumlarının yönetildiği birimdir.

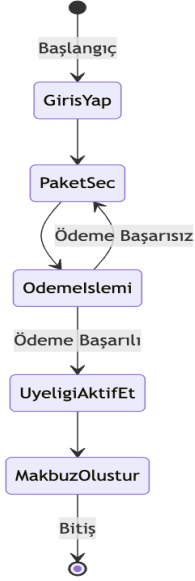
2.2. Donanım/Yazılım Eşlemesi

Geliştirilen otomasyon sistemi masaüstü ortamında çalışacak şekilde tasarlanmıştır. Sistem, **Python** programlama dili kullanılarak geliştirilmektedir. Arayüz için Python destekli modern bir GUI kütüphanesi kullanılacak olup, veriler yerel veya uzak bir **İlişkisel Veritabanı Yönetim Sisteminde (RDBMS - MySQL veya SQLite)** saklanacaktır.

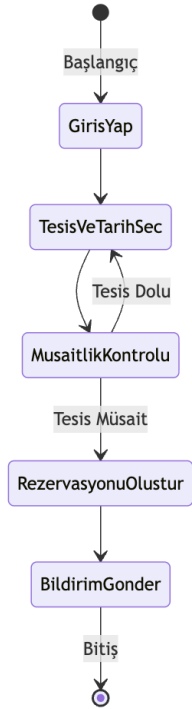
2.3. Kalıcı Veri Yönetimi

Sistemdeki tüm nesnelerin kalıcı olarak saklanması için ilişkisel bir veritabanı şeması tasarlanmıştır. Veritabanında tablolar, alan nesnelere (Kullanıcı, Uye, Rezervasyon, Tesis, Ders, UyelikPaketi, Odeme) birebir eşlenecek şekilde tasarlanmış ve veri erişim katmanı (Data Access Layer) üzerinden CRUD (Oluşturma, Okuma, Güncelleme, Silme) operasyonları yürütülecektir.

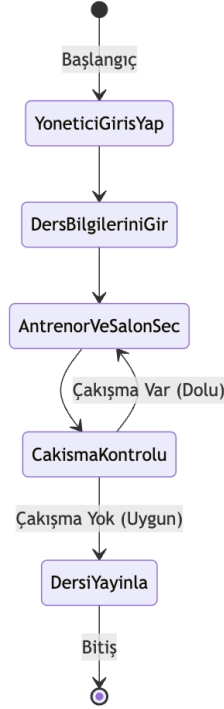
Üyelik Satın Alma Aktivite Diyagramı



Rezervasyon Yapma Aktivite Diyagramı



Ders Oluřturma Aktivite Diyagramı



2.4. Eriřim Kontrolü ve Güvenlik

Rol bazlı erişim modeli kullanılmaktadır (Üye, Yönetici, Antrenör). Kullanıcı řifreleri veritabanında düz metin (plain-text) yerine kriptografik olarak řifrelenmiş formatta saklanacaktır. Yönetici arayüzlerine, sadece Yönetici nesne tipine sahip kimlięi doęrulanmış kullanıcılar erişebilecektir.

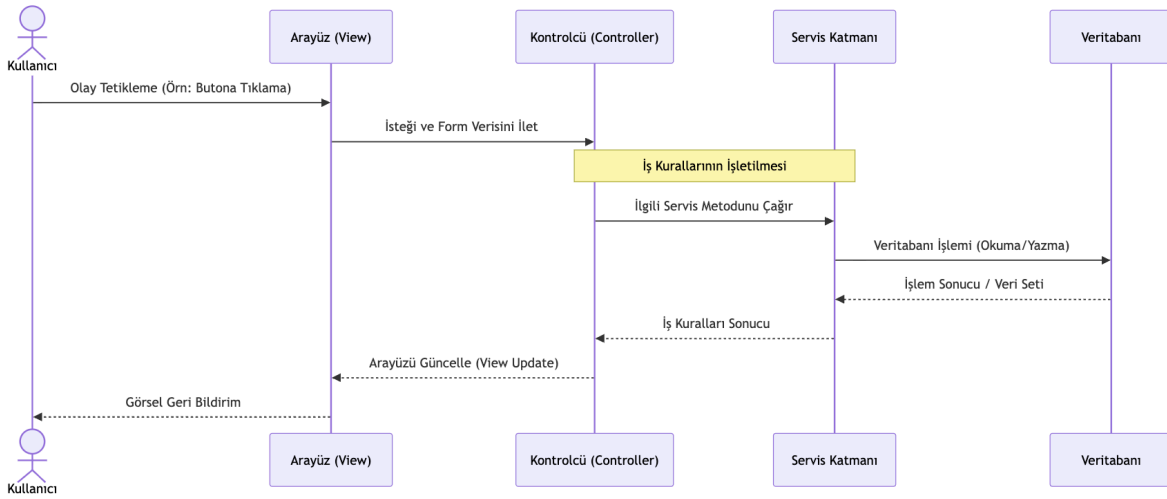
2.5. Küresel Kontrol Akışı

Sistemin kontrol akışı, kullanıcı etkileřimlerine dayalı olarak ařaęıdaki adımlarla ilerler:

1. Kullanıcı arayüz üzerinden bir işlemler başlatır (örn. rezervasyon oluřturma)

2. Bu istek Controller katmanı tarafından yakalanır
3. Controller, ilgili Service sınıfını çağırır
4. Service katmanı iş kurallarını uygular:
 - Müsaitlik kontrolü
 - Yetki kontrolü
5. İşlem uygunsa veri erişim katmanına iletilir
6. Veritabanı işlemi gerçekleştirilir
7. Sonuç tekrar Controller üzerinden kullanıcıya iletilir.

Bu yapı sayesinde sistemde gevşek bağılılık (loose coupling) ve yüksek sürdürülebilirlik sağlanmıştır.



2.6. Sınır Koşulları

2.6.1. Başlatma

Uygulama çalıştırıldığında ilk olarak konfigürasyon dosyaları okunur, veritabanı bağlantısı sınanır ve başarılıysa kimlik doğrulama ekranı (Giriş Yap) yüklenerek kullanıcıdan bilgi beklenir.

2.6.2. Sonlandırma

Kullanıcı "Çıkış Yap" eylemini tetiklediğinde, aktif oturum (session) temizlenir, kaydedilmemiş form verileri için uyarı verilir ve veritabanı bağlantısı güvenli bir şekilde kapatılır.

2.6.3. Hata Durumları

Ağ kesintisi veya veritabanı ulaşılamama durumlarında sistem exception handling yapıları ile istisnayı yakalayıp kullanıcıya anlaşılır bir hata mesajı sunar ve veritabanı transaction'larını geri alarak (rollback) veri tutarsızlığını engeller.

3. Alt Düzey Tasarım

3.1. Nesne Tasarım Kararları

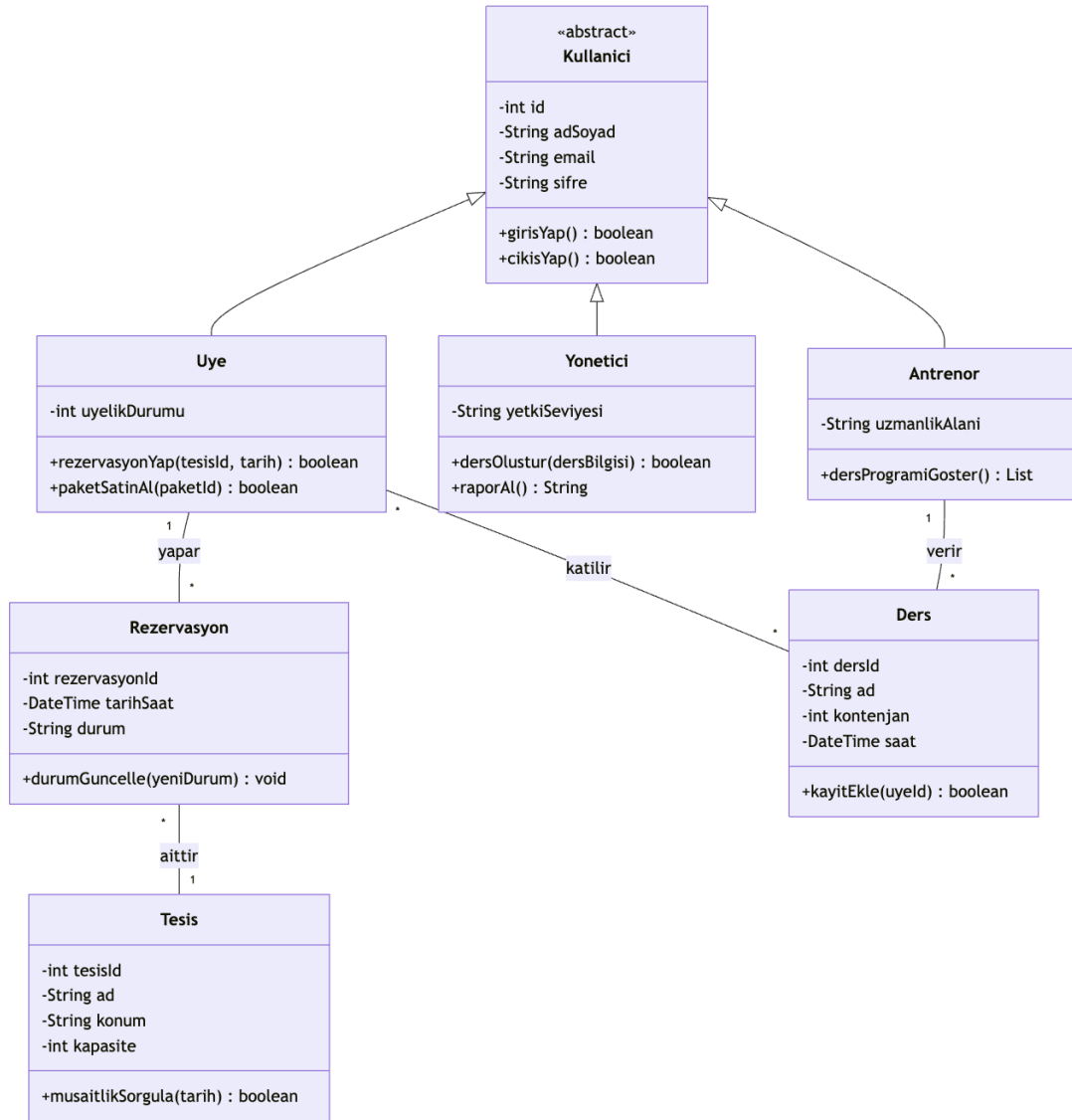
Nesne tabanlı tasarım prensiplerine sadık kalınarak; Kapsülleme (Encapsulation) kuralları gereği sınıf özelliklerine (attribute) doğrudan erişim kısıtlanmış, veri erişimi metotlar üzerinden sağlanmıştır. Modülerliği artırmak için sistem arayüzleri, servis katmanı ve veri erişim katmanı net sınırlarla ayrıştırılmıştır. Bu tasarım kararları doğrultusunda oluşturulan sınıf yapısı bir sonraki bölümde detaylı olarak sunulmuştur.

3.2. Son Nesne Tasarımı

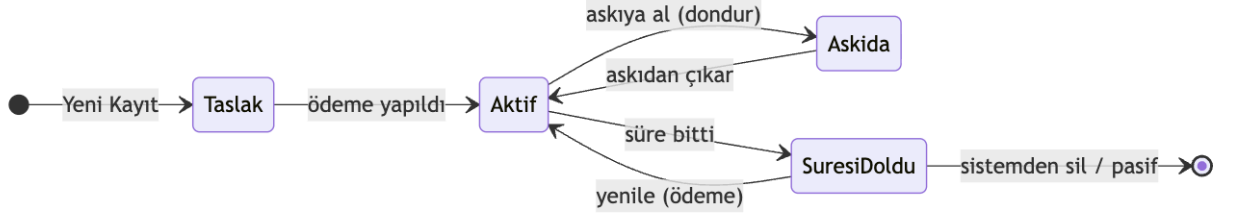
Sistemin kalbini oluşturan temel varlık (entity) sınıfları şunlardır:

- **Kullanıcı (Soyut Sınıf):** id, adSoyad, email, sifre özelliklerini ve girisYap(), cikisYap() operasyonlarını barındırır. Üye, Yönetici ve Antrenör buradan türer.
- **Tesis:** tesisId, ad, konum, kapasite özelliklerine sahiptir.
- **Rezervasyon:** rezervasyonId, tarihSaat ve durum (Taslak, Onaylandı, İptal, Tamamlandı) özelliklerine sahiptir.
- **Ders:** dersId, ad, kontenjan, saat bilgilerini tutarak antrenörler ve üyelerle ilişkilendirilir.

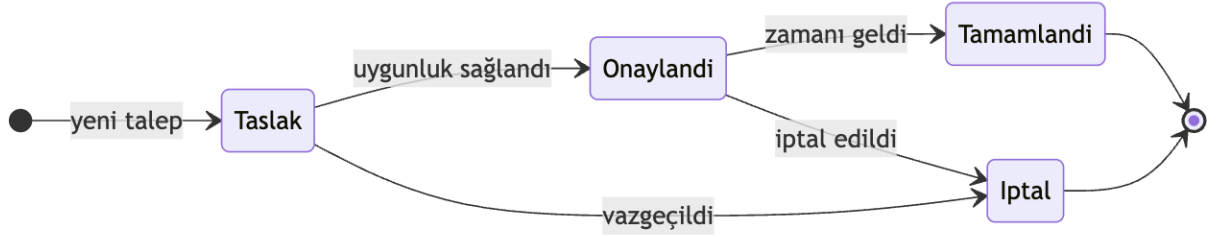
Sınıf Diyagramı:



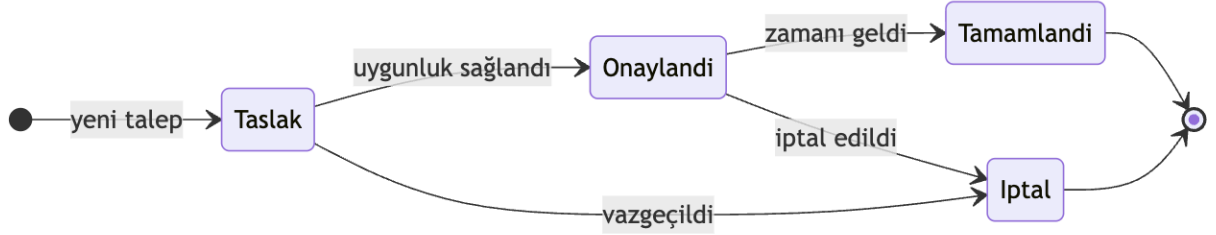
Üyelik Durum Diyagramı:



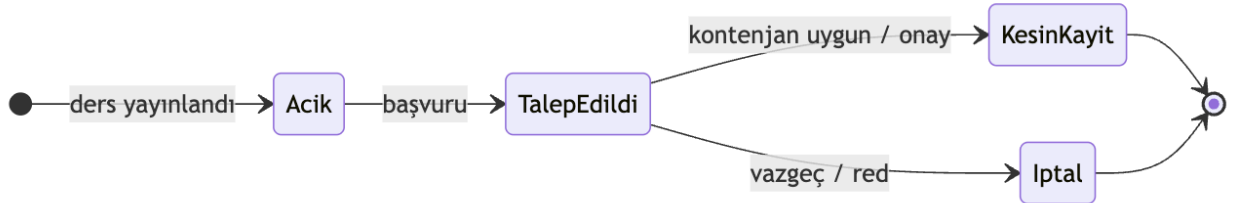
Rezervasyon Durum Diyagramı:



Ödeme Durum Diyagramı:



Ders Katılım Durum Diyagramı:



3.3. Paketler / Katmanlar

3.3.1. Dış Paketler

Sistem geliştirilirken Python standart kütüphaneleri ve arayüz bileşenleri için dış kütüphaneler (örn. PyQt, Tkinter) ile veritabanı sürücülerini kullanılacaktır.

3.3.2. İç Paketler

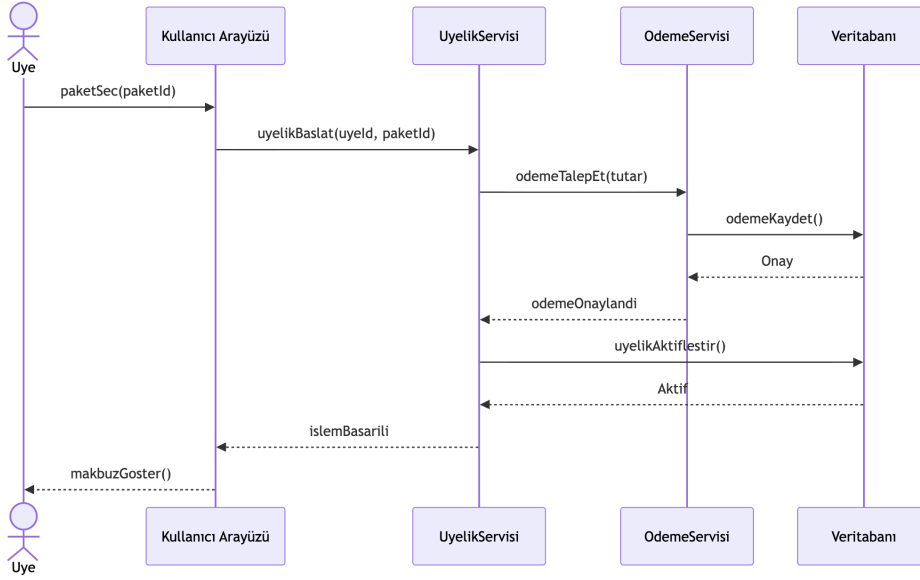
Sistem temel olarak şu paketlere ayrılır: models (varlık sınıfları), views (kullanıcı arayüzleri), controllers (iş mantığı ve form etkileşimleri), services (müsaitlik, doğrulama gibi ara işlemler).

3.4. Sınıf Arayüzleri

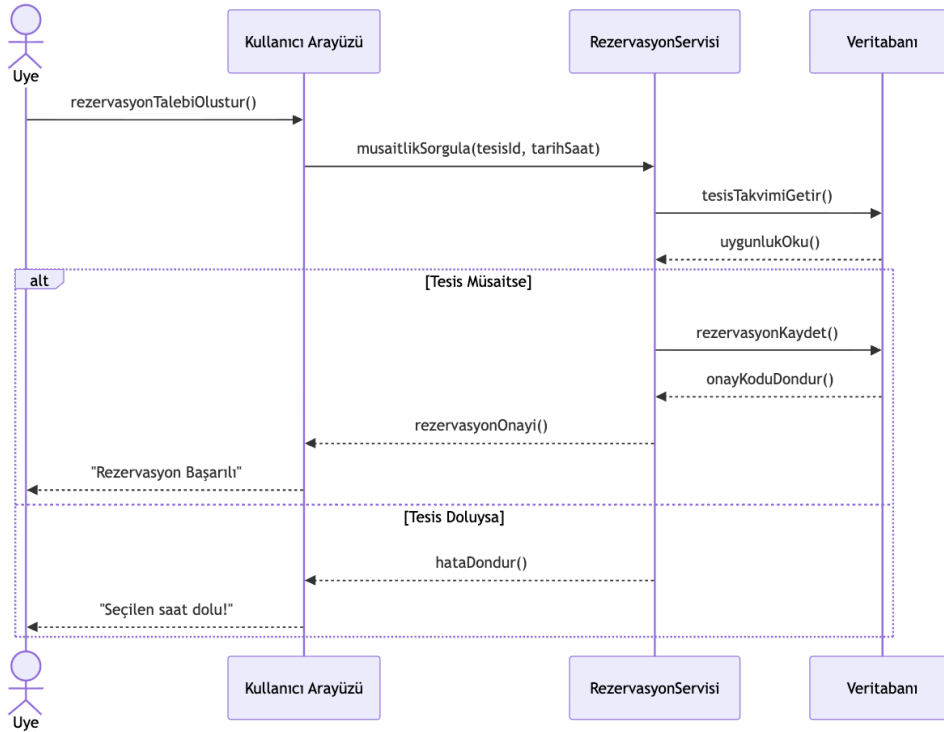
Aşağıdaki tabloda kritik bazı servis sınıflarının arayüz tanımları verilmiştir.

Sınıf / Metot	Parametreler	Dönüş Değeri	Açıklama
RezervasyonServisi.musaitlikSorgula()	tesisId: int, tarihSaat: DateTime	boolean	Belirtilen tarih ve saatte tesisin boş olup olmadığını kontrol eder.
UyelikServisi.uyelikBaslat()	uyeliId: int, paketId: int	boolean	Üye için seçilen paketi aktif hale getirme sürecini başlatır.
OdemeServisi.odemeAl()	tutar: float, uyeliId: int	OdemeDurumu	Belirlenen tutarın tahsilat sürecini yönetir.

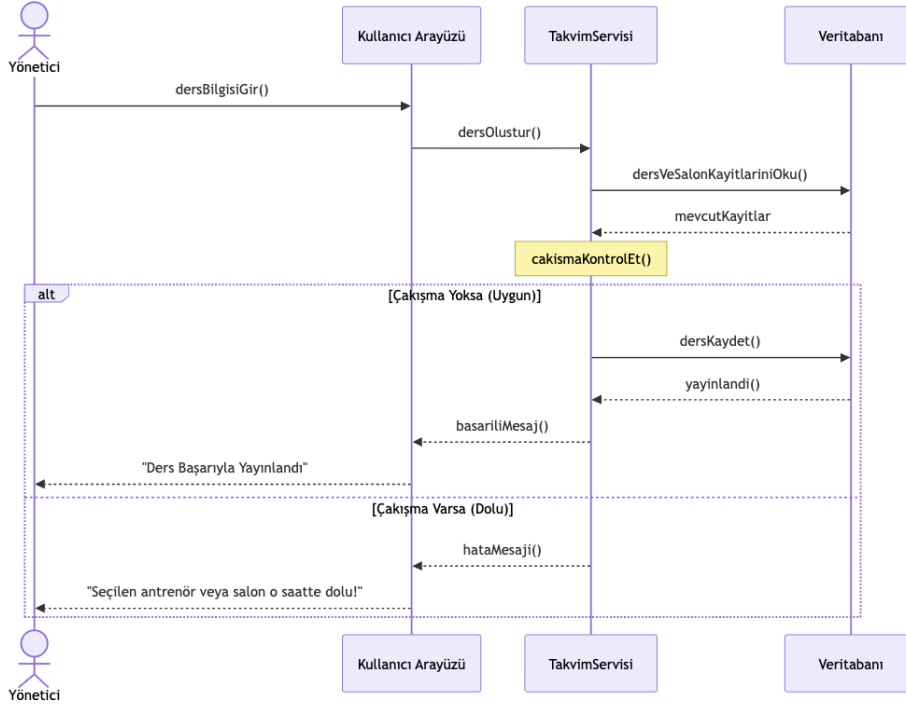
Üyelik Satın Alma - Sequence Diagram



Rezervasyon Yapma - Sequence Diagram



Ders Oluşturma - Sequence Diagram



3.5. Tasarım Desenleri

- **MVC (Model-View-Controller) / Katmanlı Mimari:** Sistem bütünüyle arayüz ve verinin ayrıştırıldığı bu genel mimari desen etrafında şekillenmiştir.
- **Singleton (Tekil):** Veritabanı bağlantı havuzunun (Database Connection) sistem genelinde sadece tek bir instance üzerinden yönetilmesi için kullanılmıştır.
- **Facade (Ön Yüz):** Karmaşık ödeme ve üyelik aktiveleştirme alt işlemlerini tek bir metod çağırısı arkasına saklamak için servis katmanında uygulanmıştır.

Sözlük (Glossary)

Terim	Tanım
Rezervasyon Çakışması	Aynı tesisin veya sahanın, aynı tarih ve saat dilimi için birden fazla kullanıcı tarafından eşzamanlı olarak kiralanmaya çalışılması durumu.
Varlık (Entity) Sınıfı	Sistemin iş mantığını temsil eden ve genellikle veritabanında bir tabloya karşılık gelen kalıcı nesneler (Örn: Üye, Tesis).
CRUD	Create, Read, Update, Delete (Oluştur, Oku, Güncelle, Sil) operasyonlarının baş harflerinden oluşan veritabanı işleme standardı.

Referanslar

- Necmettin Erbakan Üniversitesi BİL 204 Yazılım Mühendisliği Ders Notları
- Datariders, "Spor Salonu ve Tesis Yönetim Sistemi Proje Analiz Raporu"
- Datariders, "Spor Salonu ve Tesis Yönetim Sistemi Proje Özeti"
- Python Software Foundation. "Python Language Reference."